

TÀI LIỆU KỸ THUẬT · BUILD YOUR SYSTEM

Dây chuyền 4 agent Claude Code cho hệ thống ERP thật

Cách tôi chia AI thành 4 vai (lập kế hoạch, viết code, kiểm thử, soát bài), bàn giao cho nhau qua một cuốn sổ, và tự khoá tay chúng lại để chạy qua đêm mà không phá hệ thống đang vận hành.

ShinBaby ERP

Claude Code · Opus + Sonnet

Phiên bản 18/09/2026

NỘI DUNG

1. Vì sao một AI làm hết là không đủ
2. Bức tranh tổng thể
3. Sổ bàn giao — cách 4 agent nói chuyện
4. Chân dung từng agent
5. Quy trình đầy đủ từ lệnh tới commit
6. Vòng sửa và các điểm dừng
7. Hàng rào: hook chặn lệnh nguy hiểm
8. Cài đặt và vị trí file
9. Dừng hằng ngày và chạy qua đêm
10. Giới hạn — những gì dây chuyền không làm thay bạn
11. Phụ lục: mẫu file và bảng tra

1 Vì sao một AI làm hết là không đủ

Tôi tự viết hệ thống quản trị cho ShinBaby cùng AI. Không có đội kỹ thuật, không có ai review code. Một ngày làm việc bình thường của tôi với AI trông như thế này:

Tôi gõ yêu cầu, AI đọc vài file, AI viết code ngay, tôi đọc lại, tôi thấy nó quên một luật đã chốt từ tháng trước, tôi nhắc, nó sửa, tôi bảo chạy test, nó chạy cả bộ test làm máy 8GB treo, tôi dừng nó lại, tôi bảo chạy hẹp thôi, nó chạy, test xanh, tôi đẩy lên, rồi build trên máy chủ mới đổ vì một lỗi mà test không bao giờ bắt được.

Vấn đề không nằm ở chỗ AI viết code kém. Vấn đề là **một phiên AI vừa lập kế hoạch, vừa viết, vừa tự chấm bài của chính nó**. Nó không có lý do gì để nghi ngờ code nó vừa viết. Và nó phải nhớ quá nhiều thứ cùng lúc nên luật nào cũng dễ rơi.

Dây chuyền 4 agent giải quyết đúng chỗ đó: **tách vai**. Người lập kế hoạch không viết code. Người viết code không tự chấm. Người kiểm thử không được sửa code. Người soát cuối chỉ được đọc. Mỗi vai chỉ phải giỏi một việc, và vai sau luôn có quyền nói "chưa được" với vai trước.

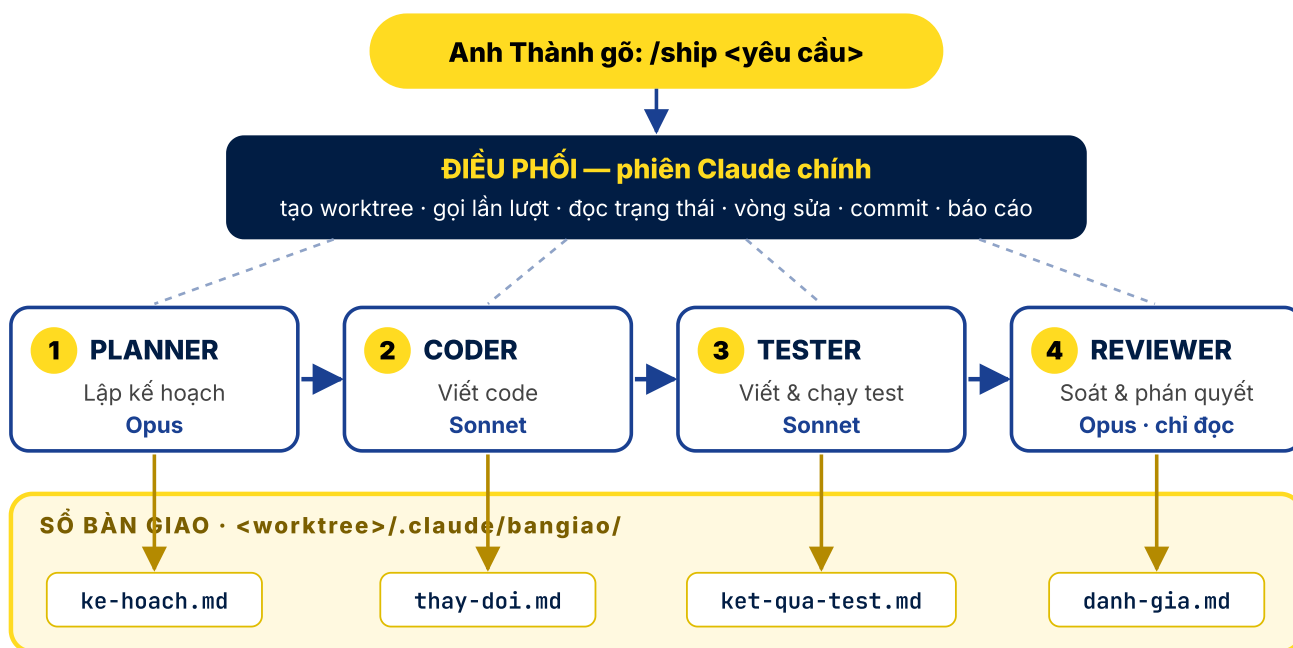
Ý TƯỞNG GỐC

Mô hình Planner → Coder → Tester → Reviewer giao việc qua một thư mục chung tôi lấy cảm hứng từ một video của kênh Cường Mê AI. Phần tôi làm thêm là gắn vào đó toàn bộ luật riêng của hệ thống ShinBaby: mỗi việc một thư mục riêng, không build trên máy yếu, môi trường máy dev trò thăng dũ liệu thật, và không bao giờ tự đẩy code lên bản đang chạy.

Nhưng đừng hiểu lầm ý của tôi: dây chuyền không làm AI thông minh hơn. Nó chỉ làm cho **cái sai lộ ra sớm hơn**, ở một chỗ có người chặn lại, thay vì lộ ra khi nhân viên cửa hàng đang bấm nút.

2 Bức tranh tổng thể

Có 5 thành phần. Phiên Claude chính đóng vai **người điều phối**: nó không viết code, chỉ gọi lần lượt 4 agent, đọc kết quả của từng bước, quyết định đi tiếp hay quay lại. Bốn agent không nói chuyện trực tiếp với nhau. Chúng để lại file trong **sổ bàn giao**, agent sau đọc file của agent trước — giống ca trực nối ca.



Hình 1. Điều phối gọi 4 agent theo thứ tự. Mỗi agent chỉ ghi đúng một file vào sổ bàn giao; agent sau đọc toàn bộ các file trước nó.

Vì sao chia model như vậy

Opus cho việc NGHĨ và SOÁT

Planner và Reviewer là hai điểm quyết định chất lượng. Kế hoạch chuẩn thì các bước sau mới ngon; người soát cuối phải đủ tinh để bắt lỗi về tiền, về quyền, về dữ liệu. Chỗ này đáng trả tiền cho model mạnh nhất.

Sonnet cho việc LÀM

Coder và Tester làm theo một bản kế hoạch đã có sẵn — đường đi nước bước đã rõ, không cần model đắt nhất. Hai vai này cũng là hai vai có thể phải chạy lại nhiều vòng, nên dùng model rẻ hơn là hợp lý.

3 Sổ bàn giao — cách 4 agent nói chuyện

Đây là mấu chốt quan trọng nhất của cả cấu trúc. Nếu các agent truyền nhau bằng lời tóm tắt, mỗi lần truyền là một lần rơi chữ. Còn khi truyền bằng file, agent nào cũng đọc được **nguyên văn** những gì các agent trước đã viết, và bạn — người chủ — mở ra là thấy toàn bộ quá trình suy nghĩ của dây chuyền.

```
# mỗi việc một worktree riêng, sổ nằm trong đó
~/shinbaby-wt/<tên-việc>/
├── .claude/bangiao/          # .claude/ đã nằm trong .gitignore → sổ không bao
giờ lọt vào commit
│   ├── yeu-cau.md           # Điều phối: yêu cầu nguyên văn, giờ bắt đầu, câu
trả lời của anh
│   ├── ke-hoach.md          # ① Planner
│   ├── thay-doi.md          # ② Coder – mỗi lượt sửa GHI THÊM một mục ở cuối
│   ├── ket-qua-test.md      # ③ Tester – mỗi lượt GHI THÊM
│   ├── danh-gia.md          # ④ Reviewer – mỗi lượt GHI THÊM
│   └── _cu/<yyyymmdd-hhmm>/  # sổ của lần /ship trước, cất đi khi có yêu cầu
mới
```

File	Ai ghi	Ai đọc	Dòng trạng thái
yeu-cau.md	Điều phối	Cả 4 agent (qua prompt)	—
ke-hoach.md	Planner	Coder, Tester, Reviewer	XONG CẦN HỎI
thay-doi.md	Coder	Tester, Reviewer, Điều phối	XONG KẾT
ket-qua-test.md	Tester	Coder (lượt sửa), Reviewer	ĐẠT RỚT
danh-gia.md	Reviewer	Coder (lượt sửa), Điều phối	CHỐT CẦN SỬA CHẶN

LUẬT CỦA ĐIỀU PHỐI

Điều phối **đọc dòng trạng thái trong file**, không tin câu trả lời miệng của agent. Agent có thể tóm tắt lạc quan; file thì ghi đúng những gì nó đã làm và kết quả lệnh đã chạy.

4 Chân dung từng agent

Mỗi agent là một file `.md` gồm phần khai báo (tên, model, công cụ được dùng, hook chặn lệnh) và phần hướng dẫn công việc. Dưới đây là những gì từng agent đọc, làm, ghi, và bị cấm.

1 Planner — người lập kế hoạch

OPUS

NHIỆM VỤ	Biến yêu cầu thường là sơ sài thành bản kế hoạch kỹ thuật đủ chi tiết để Coder làm theo mà không phải đoán.
ĐỌC	Toàn bộ <code>CLAUDE.md</code> (luật đã chốt của hệ thống) trước tiên; code liên quan; các file migration để biết bảng, cột, ràng buộc của cơ sở dữ liệu.
GHI	<code>ke-hoach.md</code> : hiểu yêu cầu · câu hỏi chặn (kèm 2–3 phương án) · giả định tự chốt · hiện trạng code · thiết kế (file, migration, quyền, menu) · bẫy áp dụng cho việc này · kế hoạch test · phạm vi không làm · tiêu chí "xong khi".
TRẢ VỀ	XONG hoặc CẦN HỎI — điều phối hỏi anh rồi gọi lại Planner.
BỊ CẤM	Ghi bất kỳ file nào khác ngoài <code>ke-hoach.md</code> ; mọi lệnh git ghi; chạy script, gọi API.

2

Coder — người viết code

SONNET

NHIỆM VỤ

Viết code đúng theo kế hoạch, không tự mở rộng phạm vi. Lướt sửa thì chỉ sửa đúng những điểm Tester hoặc Reviewer nêu.

LÀM

- Theo luật repo: logic mới ở `lib/<module>/`, client có kiểu, phân trang khi có thể quá 1.000 dòng, lọc chi nhánh, mỗi route tự kiểm quyền.
- Migration chỉ **viết ra file**, không áp vào cơ sở dữ liệu.
- Kiểm kiểu **hẹp**: chỉ những file vừa sửa, kẹp bộ nhớ 2GB.

GHI

`thay-doi.md` : danh sách file đã đổi (chính là danh sách để commit) · migration chưa áp · chỗ lệch khỏi kế hoạch và lý do · kết quả kiểm kiểu · chỗ còn hờ.

TRẢ VỀ

XONG hoặc **KẾT** (cần thư viện mới, kế hoạch sai với code thật...) → dây chuyền dừng.

BỊ CẤM

Sửa `package.json`, cài gói, mọi lệnh git ghi, sửa file bàn giao của agent khác, sửa test gác để "lách luật", bump phiên bản.

3

Tester — người kiểm thử

SONNET

NHIỆM VỤ

Viết test, chạy test, báo kết quả thật. Rớt thì báo — **không tự sửa code nghiệp vụ**.

LÀM

- Viết test vitest vào `__tests__` : đường chính, biên (rỗng, 0, âm, trùng, quá 1.000 dòng), sai quyền, sai chi nhánh.
- Chỉ chạy **file test vừa viết** + bộ test gác `__tests__/guards` + kiểm kiểu hẹp.
- Soát bằng mắt lớp lỗi test không bắt được: ràng buộc Postgres trong migration, component phía trình duyệt kéo code máy chủ.

GHI

`ket-qua-test.md` : dán **nguyên văn** dòng tổng kết của vitest/tsc; lỗi kèm `file:dòng` ; mục "Cần anh chạy thử thật" cho việc phải chạy trên dữ liệu thật.

TRẢ VỀ

ĐẠT → Reviewer · **RỚT** → quay lại Coder.

BỊ CẤM

Ghi ngoài `__tests__` ; sửa test gác; chạy toàn bộ test, build, script dữ liệu; báo ĐẠT khi chưa thật sự chạy lệnh.

4

Reviewer — người soát cuối

OPUS · CHỈ ĐỌC

NHIỆM VỤ

Người duy nhất soát code trước khi nó được commit. Khắt khe đúng mức: bắt lỗi thật, không vẽ thêm việc.

SOÁT

Theo thứ tự: đúng & đủ yêu cầu → tiền và VAT → quyền và chi nhánh → bẫy làm gãy build → migration (áp trước hay sau code) → test có kiểm thật không → chữ hiện cho nhân viên → comment.

GHI

`danh-gia.md` : mỗi lỗi phải chỉ ra `file:dòng` và *hông thể nào trong thực tế* · gợi ý không chặn · **việc anh phải làm khi deploy** · **danh sách file đưa vào commit**.

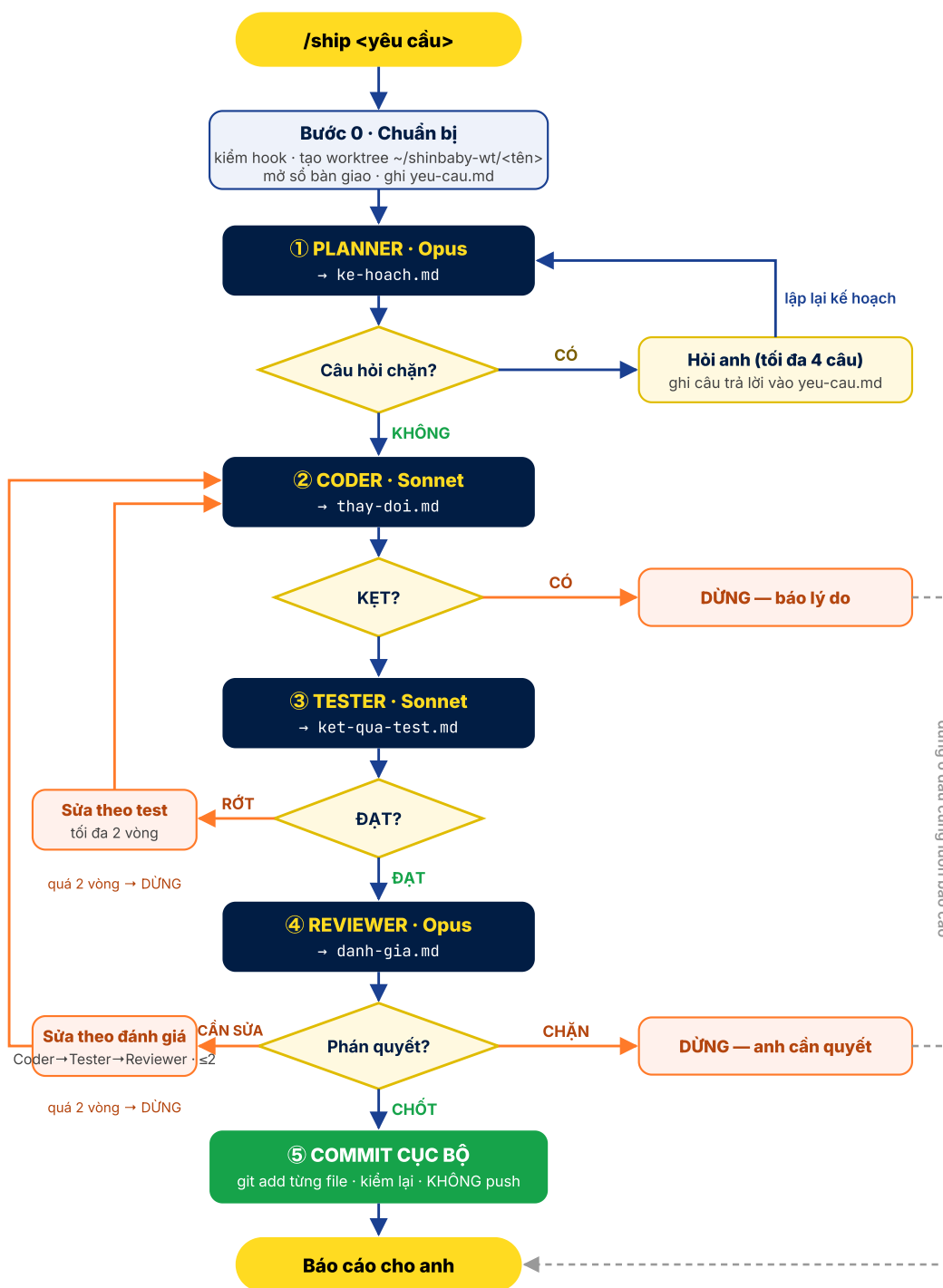
PHÁN

CHỐT không còn lỗi nặng/vừa · **CẦN SỬA** quay lại Coder · **CHẶN** sai hướng hoặc cần anh quyết.

BỊ CẤM

Sửa bất kỳ file nào ngoài `danh-gia.md` ; chạy test, build, script.

5 Quy trình đầy đủ từ lệnh tới commit



Hình 2. Toàn bộ quy trình. Đường cam là các vòng sửa, có giới hạn; đường xám là mọi lối thoát đều dẫn về báo cáo.

Diễn giải từng bước

0 Chuẩn bị

Điều phối kiểm hook chặn lệnh đã cài chưa — thiếu thì dừng ngay. Sau đó chọn thư mục làm việc: nếu phiên đang đứng trong một worktree thì dùng luôn; nếu không thì tạo worktree mới `~/shinbaby-wt/<tên>` tách từ bản đang chạy thật. Mở sổ bàn giao, ghi yêu cầu nguyên văn. Có sổ cũ thì cất vào `_cu/`. Gõ `/ship tiep` thì đọc sổ hiện có và đi tiếp từ chỗ đang dở.

1 Planner lập kế hoạch

Có câu hỏi chặn thì điều phối hỏi anh, ghi câu trả lời vào `yeu-cau.md`, rồi gọi Planner lập lại. Anh không có mặt thì câu hỏi cứ treo đó — **đúng ý**, dây chuyền không đoán thay anh những chỗ dừng tới nghiệp vụ.

2 Coder viết code

Làm theo kế hoạch, kiểm kiểu hẹp, ghi danh sách file. Trả KẾT là dừng: thường vì cần thư viện mới (cài gói bị cấm tuyệt đối) hoặc kế hoạch không khớp với code thật.

3 Tester kiểm thử

RỚT thì Coder sửa theo test, Tester chạy lại. Tối đa 2 vòng sửa; vẫn rớt là dừng và báo — không vòng vô hạn.

4 Reviewer phán quyết

CẦN SỬA thì chạy lại cả chuỗi Coder → Tester → Reviewer, tối đa 2 vòng. CHẶN thì dừng và nói rõ anh cần quyết điều gì.

5 Commit cục bộ

Lấy danh sách file trong `danh-gia.md`, đối chiếu với `git status`: file thừa không add, file thiếu ghi vào báo cáo. `git add` **từng file một** — không bao giờ add cả thư mục. Kiểm `git diff --cached` khớp đúng danh sách, commit, rồi kiểm lại lần cuối bằng `git show`. Không push, không áp migration, không deploy.

6 Vòng sửa và các điểm dừng

Một dây chuyền chạy không người trông mà không có điểm dừng thì nguy hiểm hơn là không có dây chuyền. Nó có thể sửa đi sửa lại một lỗi cả đêm, hoặc tệ hơn, "sửa" bằng cách nới lỏng chính cái test đang bắt lỗi. Vì vậy mọi lỗi ra đều được định nghĩa trước:

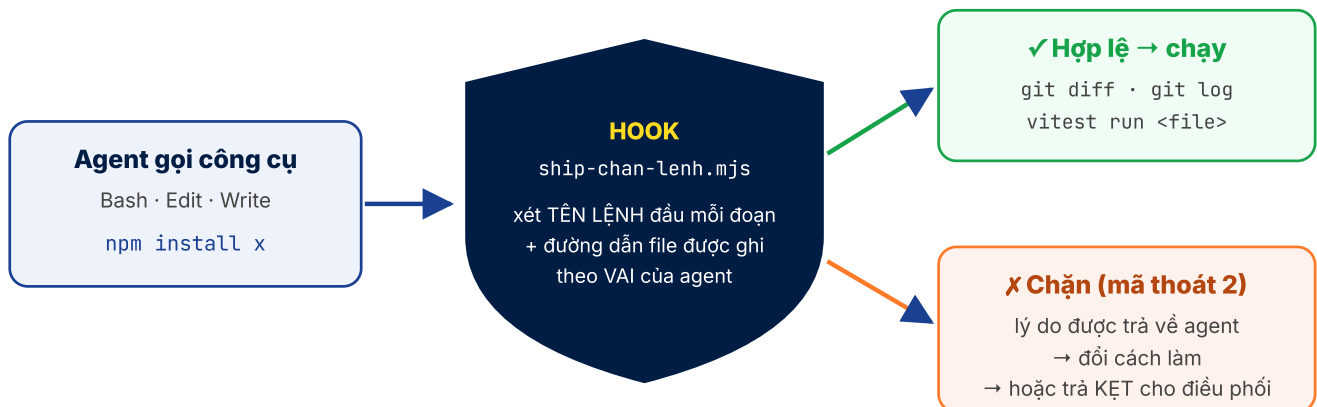
Tình huống	Ai phát hiện	Dây chuyền làm gì
Chưa cài hook	Điều phối (bước 0)	Dừng, bảo anh chạy <code>cai-dat.sh</code>
Worktree đang có việc dở không thuộc /ship	Điều phối (bước 0)	Dừng, hỏi anh — không trộn với việc dở
Yêu cầu có chỗ phải anh quyết	Planner	Hỏi anh, chờ trả lời rồi lập lại kế hoạch
Cần thư viện mới / kế hoạch sai	Coder	KẾT → dừng, báo lý do
Test rớt	Tester	Coder sửa → Tester chạy lại, tối đa 2 vòng
Lỗi sửa được trong phạm vi	Reviewer	CẦN SỬA → cả chuỗi chạy lại, tối đa 2 vòng
Sai hướng / đụng tiền, quyền cần anh quyết	Reviewer	CHẶN → dừng, nêu điều anh cần quyết
Agent chạy lệnh cấm	Hook	Lệnh bị chặn, agent nhận lý do và phải đổi cách hoặc trả KẾT

VÌ SAO CHẶN CẢ VIỆC SỬA TEST GÁC

Repo có bộ test gác (`__tests__/guards`) bắt những lỗi đã từng làm gây bản chạy thật — ví dụ số chỗ dùng kiểu `any` chỉ được giảm, không được tăng. Một AI bị ép "phải xanh" rất dễ chọn cách nhanh nhất: sửa ảnh chụp mốc của test gác cho khớp. Cả Coder lẫn Tester đều bị hook cấm ghi vào thư mục đó.

7 Hàng rào: hook chặn lệnh nguy hiểm

Dặn trong hướng dẫn là lời dặn — agent có thể quên, đặc biệt khi đang cố làm test xanh lúc 3 giờ sáng. Vì vậy mỗi lệnh của cả 4 agent đều phải đi qua một **hook** (một đoạn script chạy trước mỗi lần agent dùng công cụ) tên `ship-chan-lenh.mjs`. Hook trả mã thoát 2 là lệnh bị chặn — kể cả khi phiên đang ở chế độ bỏ qua xin phép.



Hình 3. Hook đứng giữa agent và công cụ. Nó xét tên lệnh chứ không xét chữ — nên `grep "git push"` vẫn chạy, còn `git push` thì bị chặn.

Mỗi luật chặn là một sự cố đã xảy ra thật

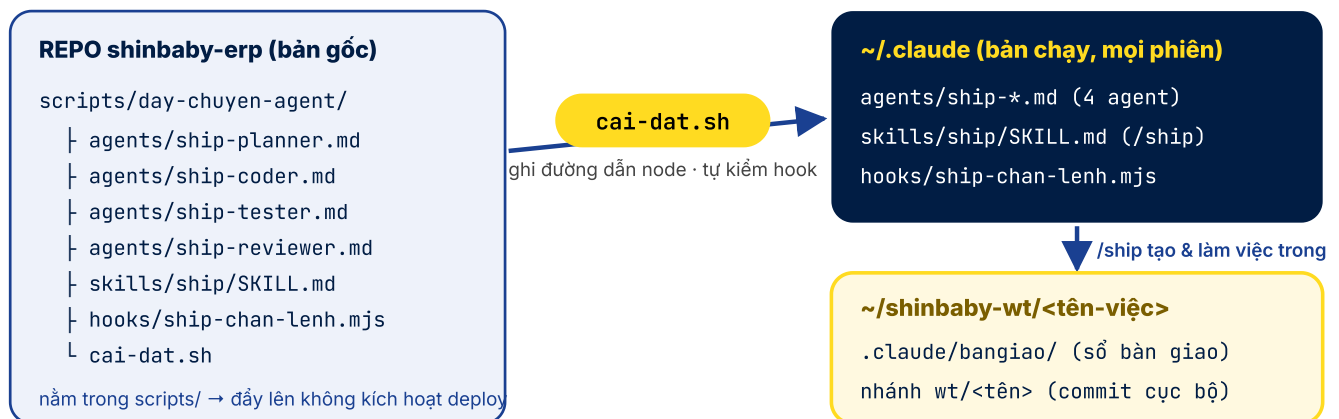
Bị chặn	Vì sao — chuyện đã xảy ra
<code>git push/commit/add/reset...</code>	Hai phiên AI cùng dùng một thư mục đã có lần commit nuốt mất 10 file code mà kiểm tra tự động vẫn báo xanh. Phần git chỉ điều phối được làm, theo đúng quy trình add từng file.
<code>npm install / ci</code>	Thư viện của mọi worktree dùng chung một chỗ. Một lệnh cài gói từng làm hỏng thư viện của 8 worktree khác cùng lúc.
build, tsc toàn dự án, chạy toàn bộ test	Máy dev chỉ có 8GB RAM; tsc toàn dự án cần khoảng 3,5GB, build làm máy swap và treo đúng lúc tôi đang dùng.
chạy script dữ liệu, <code>--apply</code> , đọc <code>.env</code> , gọi API ghi	Môi trường máy dev trở thành cơ sở dữ liệu, email, Zalo thật . Từng có một lần chạy thử gửi thư nhắc họp thật tới 5 nhân viên.
<code>vercel, ssh, pm2, gh</code>	Đựng máy chủ và deploy là việc của người, không phải của dây chuyền.
ghi file ngoài phần của mình	Planner chỉ ghi kế hoạch, Reviewer chỉ ghi đánh giá, Tester chỉ ghi trong <code>__tests__</code> , Coder không đụng <code>package.json</code> , cấu hình Claude, test gác.

ĐÃ KIỂM CHỨNG

Hook được thử với 48 trường hợp — 11 lệnh phải cho chạy, 22 lệnh phải chặn, 15 trường hợp ghi file theo từng vai — kết quả đúng cả 48. Script cài đặt tự kiểm lại hai trường hợp mẫu mỗi lần cài.

8 Cài đặt và vị trí file

Bản gốc nằm trong repo để theo dõi lịch sử bằng git. Bản chạy được cài vào `~/.claude` — cấp người dùng — vì tôi hay mở phiên ở thư mục cha `~/Documents/SHINBABY` chứ không phải trong worktree; để trong repo thì phiên đó không nhìn thấy agent.



Hình 4. Sửa ở bản gốc → chạy `cai-dat.sh` → mở phiên mới. Đừng sửa thẳng trong `~/.claude`, lần cài sau sẽ ghi đè.

Cài lần đầu hoặc sau khi sửa

```
# từ bất kỳ worktree nào đã có commit dây chuyền
./scripts/day-chuyen-agent/cai-dat.sh

# kết quả mong đợi
· agent ship-coder
· agent ship-planner
· agent ship-reviewer
· agent ship-tester
· lệnh /ship
· hook ship-chan-lenh.mjs (node: /opt/homebrew/opt/node@22/bin/node)
✓ Xong. Mở phiên Claude mới rồi gõ: /ship <yêu cầu>
```

HAI ĐIỀU HAY QUÊN

Agent mới chỉ được nạp ở phiên mở SAU khi cài. Phiên đang mở sẵn sẽ không thấy chúng.

Nâng cấp node bằng Homebrew thì chạy lại `cai-dat.sh`. Đường dẫn node được ghi cứng lúc cài; sai đường dẫn thì hook lỗi và không chặn được gì.

9 Dừng hằng ngày và chạy qua đêm

Ba lệnh cần nhớ

Lệnh	Khi nào dừng
<code>/ship <yêu cầu></code>	Giao một việc mới. Dây chuyền tự tạo worktree riêng.
<code>/ship tiếp</code>	Đi tiếp việc đang dang dở (sau khi anh trả lời câu hỏi, hoặc sau khi dừng giữa chừng).
<code>deploy đi em</code> (trong phiên mở ở worktree)	Sau khi đã đọc báo cáo và đồng ý đưa lên bản chạy thật — theo quy trình deploy thường lệ.

Viết yêu cầu thế nào cho dây chuyền ít phải hỏi lại

Planner hỏi nhiều thường vì yêu cầu thiếu ba thứ: **ai dừng**, **màn nào**, và **xong thì nhìn thấy gì**. So sánh:

Dễ phải hỏi lại

"Thêm giới hạn đăng nhập."

Chạy thẳng được

"Trang đăng nhập nhân viên: sai mật khẩu 5 lần trong 15 phút thì khoá tài khoản 15 phút, báo nhân viên chờ và liên hệ cửa hàng trưởng. Cửa hàng trưởng mở khoá được ở màn quản lý người dùng."

Checklist trước khi để chạy qua đêm

- ☐ Máy cắm sạc, bật **giữ máy thức** trong cài đặt Code của app Claude; không gập máy.
- ☐ Yêu cầu đã ghi đủ ai dùng / màn nào / xong khi nào — câu hỏi chặn sẽ làm dây chuyền đứng chờ tới sáng.
- ☐ Gõ `/ship` ở một phiên mới (để chắc agent đã được nạp).
- ☐ Sáng ra: đọc báo cáo, rồi mở `danh-gia.md` xem mục "Việc anh phải làm khi deploy".
- ☐ Có migration thì quyết thứ tự: áp trước hay sau khi code lên.

Đọc sổ bàn giao buổi sáng theo thứ tự nào

Tôi đọc ngược: **danh-gia.md** trước (kết luận, việc phải làm khi deploy), rồi **ket-qua-test.md** (test có chạy thật không, mục "cần anh chạy thử thật"), rồi mới tới **ke-hoach.md** — đặc biệt mục "giả định em tự chốt", vì đó là chỗ AI đã quyết thay mình.

10 Giới hạn — những gì dây chuyền không làm thay bạn

Nói thẳng để không kỳ vọng sai:

- **Không deploy, không áp migration.** Hai việc đó có hiệu lực ngay trên dữ liệu thật mà nhân viên đang dùng. Người quyết là người chịu trách nhiệm.
- **Không chạy thử trên dữ liệu thật.** Lỗ lỏi nằm ở ràng buộc cơ sở dữ liệu chỉ lộ ra khi chạy thật; Tester chỉ liệt kê cần chạy gì, người chạy là anh.
- **Không kiểm kiểu toàn dự án.** Máy 8GB không gánh nổi, nên chỉ kiểm hẹp trên file đã sửa. Lỗi kiểu lan sang file khác vẫn có thể lọt.
- **Chỉ giới bằng CLAUDE.md.** Planner và Reviewer soát theo luật đã chốt trong đó. Luật nào chưa được ghi lại thì dây chuyền không biết để giữ.

Nhưng bạn cũng cần nhớ: giá trị lớn nhất của dây chuyền không phải là code viết nhanh hơn. Nó là **một quy trình có sổ sách**. Mỗi việc để lại bốn file ghi rõ ai nghĩ gì, làm gì, kiểm gì, phán gì. Đó cũng chính là tinh thần "Build Your System": việc gì làm lặp lại được thì phải có hệ thống, và hệ thống thì phải có người gác cổng.

TRẠNG THÁI NGÀY 18/09/2026

Đã dựng xong, commit `0fffd67de` trên main, hook đã thử 48 ca. Lần chạy trọn vòng đầu tiên chính là lần `/ship` thật đầu tiên. Nếu trục trặc, chỗ nghi trước tiên là hook có thật sự chạy bên trong agent không — kiểm nhanh bằng cách bảo Tester chạy `git push --dry-run` : nó phải bị chặn.

11 Phụ lục: mẫu file và bảng tra**A. Khung** `ke-hoach.md` **mà Planner phải điền**

```
# Kế hoạch: <tên việc ngắn>
## Yêu cầu nguyên văn
## Hiểu yêu cầu          ai dùng, màn nào, xong thì thấy gì
## Câu hỏi CHẶN          trống nếu không có — mỗi câu kèm 2-3 phương án
## Giả định em tự chốt    để anh gạch nếu sai
## Hiện trạng            file:dòng, luồng dữ liệu, bảng/cột
## Thiết kế              file · migration · quyền · menu · chữ cho nhân viên
## Bấy phải tránh trong việc này
## Kế hoạch test
## Không làm
## Xong khi
```

B. Khung `danh-gia.md` **của Reviewer**

```
## Lướt <n>
PHÁN QUYẾT: CHỐT | CẦN SỬA | CHẶN
### Lỗi phải sửa          [NẶNG|VỪA] file:dòng — lỗi — hỏng thế nào — sửa thế nào
### Gợi ý (không chặn)
### Việc anh phải làm khi deploy
### Danh sách file đưa vào commit
```

C. Bảng tra nhanh

Agent	Model	Công cụ	Được ghi
ship-planner	Opus	Read, Grep, Glob, Bash, Write	ke-hoach.md
ship-coder	Sonnet	Read, Grep, Glob, Edit, Write, Bash	code trong worktree + thay-doi.md
ship-tester	Sonnet	Read, Grep, Glob, Edit, Write, Bash	__tests__/ (trừ guards) + ket-qua-test.md
ship-reviewer	Opus	Read, Grep, Glob, Bash, Write	danh-gia.md

Build Your System

Việc gì làm lặp lại được thì phải có hệ thống — và hệ thống thì phải có người gác cổng.

nguyenhiepthanh.vn